

ЛАБОРАТОРНАЯ РАБОТА №13
ИСПОЛЬЗОВАНИЕ RECYCLERVIEW: АДАПТЕР, VIEWHOLDER, ОПТИМИЗАЦИЯ
СПИСКА

Что делаем сегодня?

В прошлой лабораторной работе мы познакомились с **ListView**, в этот раз разберем его современного наследника **RecyclerView**

А в этой мы напишем саму **логику** – правила, выведем слово, и тд.

Основные правила

1. Если не видно код – **ПРИБЛИЖАЙТЕ файл**
2. Обычно в конце каждой части есть **ЗАДАНИЕ** – его надо сделать
3. Не бойтесь **задавать любые** вопросы
4. Если хотите сделать что-то **свое**, то **предлагайте** – будем думать
5. Советую читать текст, тут **написано** много полезного
- 6.

ВНИМАНИЕ

Часть 1. Что такое RecyclerView

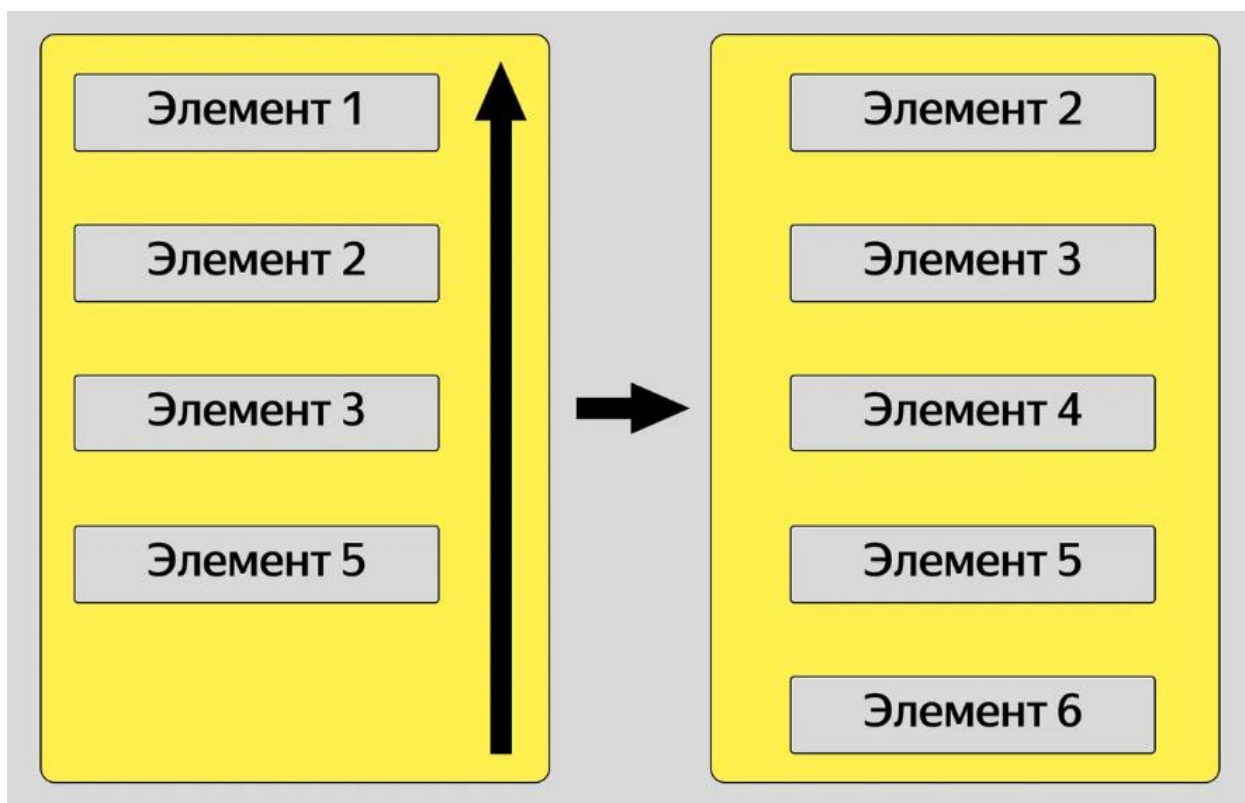
Одна из особенностей большинства устройств на Android – небольшой размер экрана. И на этом небольшом экране часто нужно отобразить **большой объём информации**: много картинок в галерее, полный каталог товаров, список музыкальных треков.

Чтобы это сделать, нужно настроить отображение по блокам, плавную **прокрутку, группировку** элементов. Кроме того, важно, чтобы объёмный список не слишком нагружал ресурсы устройства – иначе оно просто **будет зависать** (что ну очень раздражает).

Так в чем же суть RecyclerView?

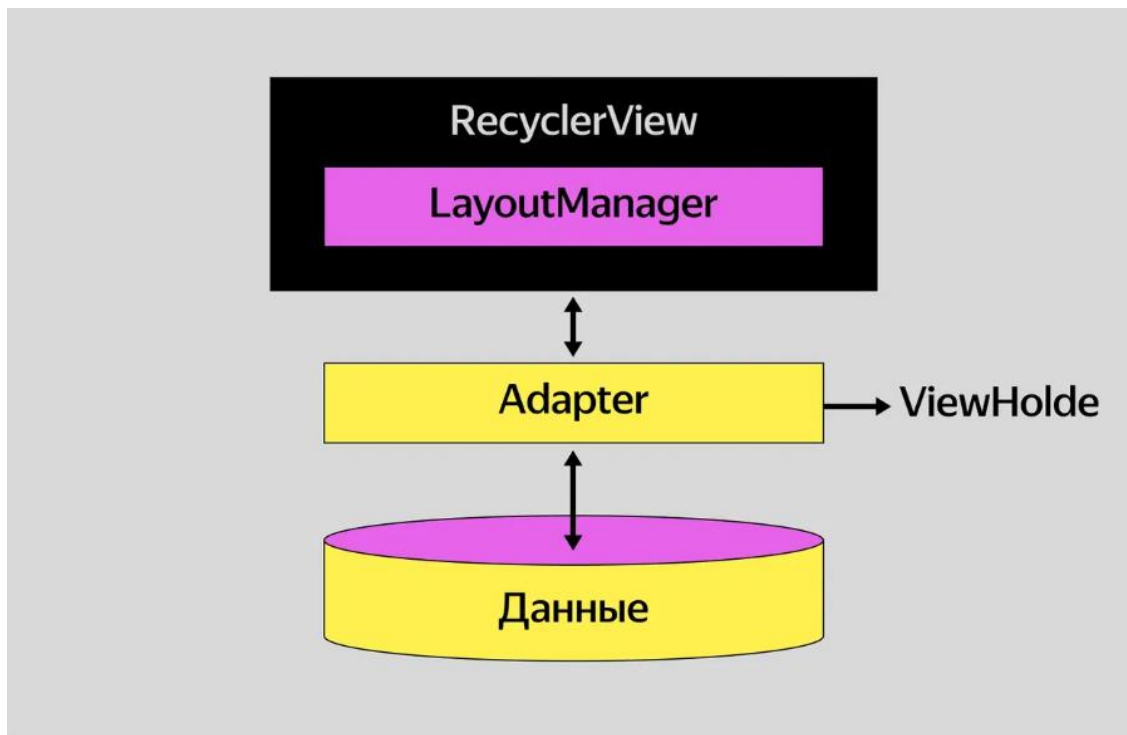
Принцип работы RecyclerView основывается на простой идее –

не нужно хранить в памяти то, что пользователь не видит в данный момент.



RecyclerView делает так, что при прокрутке элементы плавно сменяют друг друга – верхний исчезает, а нижний появляется.

Сам **RecyclerView** отвечает только за отображение элементов в гибкой и масштабируемой сетке. Все остальную работу обеспечивают дополнительные компоненты:



Теперь добавим сам **RecyclerView** в xml нашего экрана:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:id="@+id/main"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      tools:context=".MainActivity">
9
10     <androidx.recyclerview.widget.RecyclerView
11         android:id="@+id/rvProducts"
12         android:layout_width="match_parent"
13         android:layout_height="match_parent"
14         />
15
16 </LinearLayout>
```

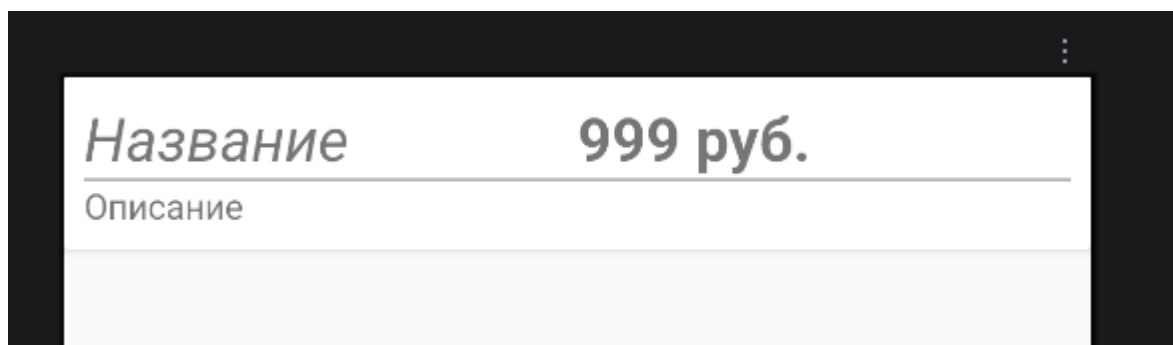
Часть 2. Создание модели и её разметки

Для начала, чтобы создать сам **СПИСОК** нам необходимо создать **модель** наших данных, которая будет описывать 1 элемент, например, **в каталоге товаров** – моделью является товар. Потому для примера создам класс **Товар**:

```
2
3 public class Product { 9 usages
4     public String name;
5     public int price; 2 usages
6     public String desc; 2 usages
7
8     public Product(String name, int price, String desc) { 5 usages
9         this.name = name;
10        this.price = price;
11        this.desc = desc;
12    }
13 }
14
```

Обязательно добавляем **КОНСТРУКТОР**, чтобы мы могли создать объект нашего класса.

Так как список состоит из повторяющихся элементов, значит, что нам надо создать разметку **ОДНОЙ строчки списка**. Создам отдельный xml файл для этого:



Название	999 руб.
Описание	

ОБЯЗАТЕЛЬНО не забываем назначить **id** на поля, которые будут меняться списком (у меня например это название, цена и описание).

Часть 3. Создание адаптера и ViewHolder

Вспоминаем, что **RecyclerView** сам по себе отвечает только за отображение – больше, по сути, он ничего не делает, ему помогают эти два героя:

1. **ViewHolder** – объект, который определяет, как именно должен выглядеть элемент списка на экране. Также он отображает передаваемые в него данные.
2. **Adapter**. Соединяет между собой сырые данные, которые нужно поместить в RecyclerView. Именно Adapter передает в RecyclerView элементы списка.

Создаем класс **Adapter** для нашего списка:

```
2
3 public class ProductAdapter {
4
5 }
6
```

Внутри него создаем класс **ViewHolder**, который будет отвечать за отображение:

```
public class ProductAdapter { no usages
    static class ProductViewHolder extends RecyclerView.ViewHolder { no usages
        TextView tvName; 1 usage
        TextView tvPrice; 1 usage
        TextView tvDesc; 1 usage

        public ProductViewHolder(@NonNull View itemView) { no usages
            super(itemView);

            tvName = itemView.findViewById(R.id.tvName);
            tvPrice = itemView.findViewById(R.id.tvPrice);
            tvDesc = itemView.findViewById(R.id.tvDesc);
        }
    }
}
```

Создаем ему поля для хранения штук с экрана, а также у него в конструкторе не забываем их найти в разметке одной строчки.

Теперь делаем наш **Adapter** наследником **RecyclerView.Adapter**, а в треугольных скобках прокидываем наш **ViewHolder**:

```
9 public class ProductAdapter extends RecyclerView.Adapter<ProductAdapter.ProductViewHolder> {
10
11     static class ProductViewHolder extends RecyclerView.ViewHolder { 1 usage
12         TextView tvName; 1 usage
13         TextView tvPrice; 1 usage
14         TextView tvDesc; 1 usage
15
16         public ProductViewHolder(@NonNull View itemView) { no usages
17             super(itemView);
18
19             tvName = itemView.findViewById(R.id.tvName);
20             tvPrice = itemView.findViewById(R.id.tvPrice);
21             tvDesc = itemView.findViewById(R.id.tvDesc);
22         }
23     }
24 }
25
```

Нажимаем **Implement methods**, чтобы андроид студия за нас создала нужные методы для **Adapter** и нажимаем там **OK**:

```
public class ProductAdapter extends RecyclerView.Adapter<ProductAdapter.ProductViewHolder> { 1 usage
    static class ProductViewHolder extends RecyclerView.ViewHolder { 1 usage
        TextView tvName; 1 usage
        TextView tvPrice; 1 usage
        TextView tvDesc; 1 usage
        public ProductViewHolder(@NonNull View itemView) {
            super(itemView);
            tvName = itemView.findViewById(R.id.tvName);
            tvPrice = itemView.findViewById(R.id.tvPrice);
            tvDesc = itemView.findViewById(R.id.tvDesc);
        }
    }
}

Implement methods
Make 'ProductAdapter' abstract
Create subclass
Create test
Remove qualifier
Replace constructor with factory method
Unimplement Class
Press Ctrl+Q to toggle preview

4 import android.os.Bundle;
11 @NonNull
12 @Override
13 public ProductAdapter() {
14     return null;
15 }
16
17 @Override
18 public void onBindViewHolder(@NonNull ProductViewHolder holder, int position) {
19 }
20 }
21
22 @Override
23 public int getItemCount() {
24     return 0;
25 }
```

По итогу получается что-то такое:

```
10 public class ProductAdapter extends RecyclerView.Adapter<ProductAdapter.ProductViewHolder> {
11
12     @NonNull
13     @Override
14     public ProductViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
15         return null;
16     }
17
18     @Override
19     public void onBindViewHolder(@NonNull ProductViewHolder holder, int position) {
20
21     }
22
23     @Override
24     public int getItemCount() {
25         return 0;
26     }
27
28     static class ProductViewHolder extends RecyclerView.ViewHolder { 3 usages
29         TextView tvName; 1 usage
30         TextView tvPrice; 1 usage
31         TextView tvDesc; 1 usage
32
33         public ProductViewHolder(@NonNull View itemView) { no usages
34             super(itemView);
35
36             tvName = itemView.findViewById(R.id.tvName);
37             tvPrice = itemView.findViewById(R.id.tvPrice);
38             tvDesc = itemView.findViewById(R.id.tvDesc);
39         }
40     }
41 }
```

Создались такие методы (пока что они пустые, **мы должны их сами написать**):

1. **onCreateViewHolder** – этот метод будет вызываться, когда списку нужно СОЗДАТЬ новую пустую карточку
2. **onBindViewHolder** – этот метод связывает данные с карточкой (проще говоря, заполняет тексты и вьюшки)
3. **getItemCount** – говорит списку, сколько всего у нас элементов

Перед этим мы добавим **поле для хранения** нашего списка и добавим **конструктор** для его заполнения:

```
11
12 public class ProductAdapter extends RecyclerView.Adapter<ProductAdapter.ProductViewHolder> {
13
14     private List<Product> products; 1 usage
15
16     public ProductAdapter(List<Product> products) { no usages
17         this.products = products;
18     }
19
20     @NonNull
21     @Override
22     public ProductViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
23         return null;
24     }
25
26     @Override
```

Теперь по очереди реализуем каждый метод:

```
@NonNull
@Override
public ProductViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    // создаем объект VIEW по нашему XML файлу
    View view = LayoutInflater.from(parent.getContext())
        .inflate(R.layout.product_row, parent, attachToRoot: false);

    // и сразу привязываем его к нашему "ДЕРЖАТЕЛЮ" (HOLDER)
    return new ProductAdapter.ProductViewHolder(view);
}
```

```
@Override
public void onBindViewHolder(@NonNull ProductViewHolder holder, int position) {
    Product product = products.get(position); // Берем товар по номеру в списке
    holder.tvName.setText(product.name);      // достаем из него имя
    holder.tvPrice.setText(String.valueOf(product.price)); // и цену тоже
    holder.tvDesc.setText(product.desc);      // достаем из него описание
}
```

```
@Override
public int getItemCount() {
    return products.size();
}
```

Часть 4. Навешиваем адаптер

Добавляем на наш экран **поле** для списка, делаем **метод для его заполнения** и вызываем **очень много раз**:

```
17 public class MainActivity extends AppCompatActivity {
18
19     private final List<Product> rows = new ArrayList<>();
20
21     @Override
22     protected void onCreate(Bundle savedInstanceState) {
23         super.onCreate(savedInstanceState);
24         EdgeToEdge.enable(this);
25         setContentView(R.layout.activity_main);
26         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {});
27
28         for (int i = 0; i < 999; i++) {
29             loadData();
30         }
31
32     private void loadData() {
33         rows.add(new Product( name: "Эспрессо", price: 100, desc: "Классика, дефолт, база, основа"));
34         rows.add(new Product( name: "Американо", price: 150, desc: "Описание 123"));
35         rows.add(new Product( name: "Капучино", price: 180, desc: "Эспрессо с молоком и пенкой"));
36         rows.add(new Product( name: "Латте", price: 200, desc: "На кокосовом"));
37         rows.add(new Product( name: "Раф", price: 250, desc: "Сливочный, с ванильным сахаром"));
38     }
39 }
```

Теперь **сделаем привязку** нашего адаптера к RecyclerView:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {});

    for (int i = 0; i < 999; i++) {
        loadData();
    }

    ProductAdapter adapter = new ProductAdapter(rows); // создаем адаптер и закидываем список
    RecyclerView rv = findViewById(R.id.rvProducts); // находим на экране рекуклервью
    rv.setLayoutManager(new LinearLayoutManager(context, this)); // делаем вертикальную прокрутку
    rv.setAdapter(adapter); // привязываем наш адаптер к списку
}
```

Готово! Если запустить, то отобразится список.

! ЗАДАНИЕ !

Сделайте свой экран с использованием **RecyclerView** и своего **Adapter** (включая **ViewHolder**) на одну из тем:

№	Тема
1	Книжный магазин
2	Каталог растений
3	Продажа вещей
4	Каталог игр
5	Список музыкальных альбомов/треков
6	ЛЮБАЯ своя тема

Требования к итоговой версии

1. Есть **класс-модель** с несколькими полями
2. Список на ~1000+ позиций прокручивается **плавно**
3. Реализован **адаптер** с использованием **ViewHolder**

Дополнительно (по желанию)

1. **Четные/нечетные.** Окрасьте фон строк через одну: в onBindViewHolder проверяйте **position % 2 == 0**.
2. **Картинка.** Добавьте в модель поле – int ImageId, которое будет отображаться в ImageView